PYTHON FUNDAMENTALS

Lesson 1.1: Variable Declaration & Naming Conventions

Chapter 1: Variables and Primitive Data Types

LEARNING OBJECTIVES

- Understand Python's object reference model and Dynamic Typing mechanism.
- Master the syntax for variable assignment, reassignment, and multiple assignments.
- Apply official PEP 8 naming conventions.
- Identify, troubleshoot, and prevent common variable initialization and naming errors.

Instructor: Mr Dam Vinh Hung| Course Code: PY101 | Version: 2026.1

CORE CONCEPTS

Variables & Dynamic Memory Allocation

What is a Variable in Python?

- A variable is a symbolic name (label) that references an object stored in memory—not a fixed memory box.
- Created automatically the moment you first assign a value using the assignment operator (=).
- Variables hold memory addresses (pointers) to heap-allocated objects.

Dynamic Typing Mechanism

- Type lives with the Object, not the Variable: No explicit type declaration is needed prior to assignment.
- Runtime Type Resolution: The Python interpreter infers data types dynamically at execution time.
- Garbage Collection: Unreferenced objects in memory are automatically reclaimed by Python's memory manager.

Memory Reference Inspection

```
# Variable 'score' binds to int object 100
score = 100
print(type(score), id(score))
# Output: <class 'int'> 1407128394

# Rebinding 'score' to a new str object
score = "A+ Distinction"
print(type(score), id(score))
# Output: <class 'str'> 2198471024
```

Key Takeaway: Reassigning a variable does not overwrite the old memory block; it creates a new object and moves the label to the new address.

SYNTAX & EXAMPLES

Variable Declaration & Assignment Syntax

Fundamental Syntax: variable_name = expression

- Left-Hand Side (LHS): Valid identifier name.
- Assignment Operator (=): Binds LHS identifier to RHS object.
- Right-Hand Side (RHS): Evaluated completely before binding occurs.

Supported Assignment Patterns

- Standard Assignment: Single variable bound to a single literal/expression.
- Chained Assignment: Bind multiple names to the exact same immutable object.
- Positional Unpacking: Assign multiple variables simultaneously in one line.
- In-Place Swapping: Exchange values cleanly without temporary variables.

Code Demonstrations

```
# 1. Standard Single Assignment
course_name = "Python Fundamentals"
credit_hours = 3
passing_gpa = 2.5
is_published = True

# 2. Chained Assignment (Shared Reference)
min_score = default_score = 0

# 3. Tuple Unpacking / Multiple Assignment
x_coord, y_coord, z_coord = 10.5, -4.2, 0.0

# 4. Pythonic Variable Swapping
left_node, right_node = "Alpha", "Beta"
left_node, right_node = right_node, left_node
```

Identifier Rules & PEP 8 Naming Conventions

Mandatory Interpreter Rules (Syntax)

- Allowed Characters: Letters (a–z, A–Z), digits (0–9), and underscores (_).
- Leading Character: Cannot start with a digit (e.g., 1st_place is invalid).
- Case Sensitivity: user_id, User_Id, and USER_ID are three distinct names.
- No Reserved Keywords: Cannot use Python's 35 reserved words (if, for, class, def, return, None, global, etc.).

PEP 8 Style Guidelines (Best Practices)

- Use snake_case for standard variables and function names.
- Use UPPER_SNAKE_CASE for module-level constants.
- Choose descriptive, intention-revealing names over single-letter abbreviations.

Naming Comparison Matrix

Category	Valid & Pythonic (PEP 8)	Invalid / Discouraged
Standard Var	`student_count`	`studentCount (camelCase)`
Boolean Flag	`is_authenticated`	`flag1 (vague meaning)`
Constant	`MAX_RETRY_LIMIT`	`max-retry (hyphen error)`
Internal / Private	`_cache_buffer`	`2nd_buffer (starts w/ digit)`
Reserved Word	`class_room_id`	`class (SyntaxError)`

Tip: Run `import keyword; print(keyword.kwlist)` in Python to inspect all 35 reserved keywords.

TROUBLESHOOTING & PITFALLS

Common Variable Mistakes & How to Fix Them

Error Type	Problematic Code (Anti-Pattern)	Corrected Code & Resolution
1. Uninitialized Variable (NameError)	print(total_price) → Accessing a variable before assignment or misspelling its identifier.	total_price = 0.0 print(total_price) → Always initialize variables prior to read operations.
2. Shadowing Built-ins (TypeError Later)	list = [10, 20, 30] new_list = list("abc") → Overwrites built-in list() constructor with a list instance.	scores_list = [10, 20, 30] chars = list("abc") → Never name variables list, str, int, id, sum, or max.
3. Unpacking Mismatch (ValueError)	a, b = 10, 20, 30 → Too many values to unpack (expected 2, got 3).	a, b, c = 10, 20, 30 (or a, *b = 10, 20, 30) → Match variable count to sequence length or use star unpacking.
4. Shared Mutable Ref (Logical Bug)	row_a = row_b = [] row_a.append(1) → Both names point to the exact same list; row_b is also [1]!	row_a, row_b = [], [] row_a.append(1) → Initialize separate mutable objects independently.

Mini Practice & Concept Checkpoint

Q1 (Multiple Choice): Which of the following variable names is BOTH syntactically valid in Python and compliant with PEP 8 guidelines?

```
A) 2nd_user_score     B) user-profile-id     C) max_connection_timeout     D) global
```

A) 2nd_user_score

B) user-profile-id

C) max_connection_timeout

D) global

Mini Practice & Concept Checkpoint

Q1 (Multiple Choice): Which of the following variable names is BOTH syntactically valid in Python and compliant with PEP 8 guidelines?

`A) 2nd_user_score` `B) user-profile-id` `C) max_connection_timeout` `D) global`

A) 2nd_user_score

B) user-profile-id

C) max_connection_timeout

D) global

Mini Practice & Concept Checkpoint

Q2: What is printed to the console after executing this snippet?

```
x = 15;  y = x;  x = 40;  print(y)
```

A) 40

B) 15

C) None

D) NameError

Mini Practice & Concept Checkpoint

Q2: What is printed to the console after executing this snippet?

```
x = 15;  y = x;  x = 40;  print(y)
```

A) 40

B) 15

C) None

D) NameError

Mini Practice & Concept Checkpoint

Buggy Code Snippet

```
# Fix 3 issues in this code:
1st_student = "Alice"
class = "PY101"
StudentAge = 20
```

THE END

THANK YOU FOR WATCHING

Primary Technical References

- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). PEP 8 – Style Guide for Python Code. Python Software Foundation.
- Python Software Foundation. The Python Language Reference (v3.12+): Execution Model & Data Model.